

Camera-in-the-Loop Optical Training of a Physical Diffractive Network on a Smartphone

Svetoch Series, Paper VI of VI

Oleg Yuryevich Kirichenko

Independent researcher

urevich55@gmail.com • GitHub: [@infosave2007](https://github.com/infosave2007)

17 June 2026

Published on Zenodo — DOI [10.5281/zenodo.20730393](https://doi.org/10.5281/zenodo.20730393)

Abstract

Paper I of this series showed that a smartphone’s OLED screen, an ordinary mirror, and the front camera evaluate the forward pass of a neural network *in light*. Here we close the loop and **train**. We treat the optical channel itself — the smartphone’s display→mirror→camera path, a **single-layer physical diffractive network** whose forward pass is carried by near-field (Talbot) diffraction and on-sensor integration — as the layer under optimization: the weights are displayed as brightness, the camera reads the channel’s true response (the real, noisy, non-linear forward pass), an error against a target is formed, and the weights are updated — then the cycle repeats. We demonstrate two on-device optical learners. (1) *Optical gradient descent*: eight weights drawn as brightness columns are optimized by the delta rule $\mathbf{w} \leftarrow \mathbf{w} - \eta(\mathbf{y} - \hat{\mathbf{y}})$ ($\eta = 0.7$) using the camera-measured output $\mathbf{y}$; over eight iterations the optically read mean-squared error falls by more than half, below a 0.05 pass threshold. (2) *An optical perceptron*: an eight-input perceptron trained for AND, OR and MAJORITY is evaluated through the optical dot product, retaining accuracy above the 0.7 threshold. We also describe a single-shot *optical loss read-out by inverted overlay*. The contribution is to show that a *trainable* optical loop — not merely inference — can be built from commodity hardware, and to delimit honestly what is optical (the forward pass and the loss read-out) and what remains on the CPU (the weight-update arithmetic).

Keywords: optical computing, in-the-loop training, physics-aware learning, hardware-in-the-loop, delta rule, perceptron, analog accelerator, smartphone optics.

1 Introduction

Inference is only half of machine learning; the other half is **training**, which is even more compute-hungry. Optical inference accelerators are well studied, but optical *training* — closing the feedback loop so the physical system improves itself — is harder: it requires reading the output, scoring it, and adjusting the inputs, all through the same imperfect channel.

Paper I established the forward primitive: a camera photosite integrates the light from many screen pixels and returns a multiply–accumulate. The natural next question is whether that same channel can sit *inside* an optimization loop. It can. Because the camera reports the channel’s **true** response — including non-linearity, vignetting and noise — a loop that minimizes the measured error learns weights that are correct *for the real optics*. This is the defining advantage of hardware-in-the-loop training: the physics is never approximated, because the physics is doing the forward pass.

We call the trainable element a *physical diffractive network*: light from the displayed weights propagates through the air gap by near-field (Talbot) diffraction, reflects, and is integrated on the sensor, so the channel realizes one optical layer whose response we shape by training. We mean this in the literal, single-layer sense — **not** a multilayer diffractive deep neural network (D²NN) in the sense of Lin *et al.*; the distinction is made explicit in Section 7.

2 The optical channel as a physical diffractive network

The trainable layer is the screen→mirror→camera channel of Paper I. Light leaving the displayed weights propagates through the air gap — where near-field (Talbot) diffraction sets the usable spatial bandwidth — reflects off the mirror, and is integrated on the sensor. That is a single physical diffractive layer: a fixed optical transform whose *inputs* (the displayed weights) we adjust by training, rather than a stack of learned phase masks.

Encode a weight vector $\mathbf{w}$ as the brightness of a row of screen columns. From Paper I, the camera’s read-out of column i against an input x_i is, after white-normalization and γ -correction, proportional to $w_i x_i$; summing a region yields the dot product. The measured output $\mathbf{y} = f_{\text{opt}}(\mathbf{w})$ is the channel’s forward pass, including every real imperfection. Training finds $\mathbf{w}$ such that $\mathbf{y}$ matches a target $\hat{\mathbf{y}}$, by iterating display → capture → score → update (Figure 1); the gradient information is a physical measurement.

Figure 15. Camera-in-the-loop optical training: the channel is the trainable layer

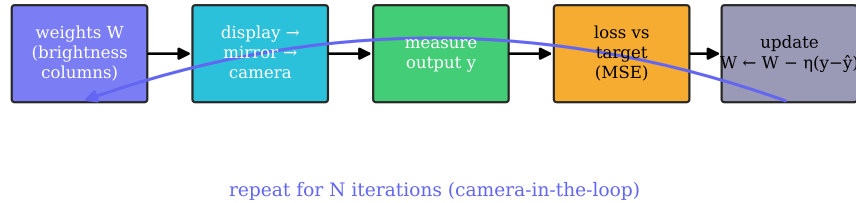


Figure 1: Camera-in-the-loop optical training. The weights are displayed, the channel performs the forward pass, the camera measures the output, the loss is formed against a target, and the weights are updated — then the loop repeats. The trainable “layer” is the real optical channel.

3 Optical gradient descent by the delta rule

In `stage41_optsgd`, eight weights are drawn as eight brightness columns. Per iteration the camera measures the eight-bin output $\mathbf{y}$, the MSE against a target $\hat{\mathbf{y}}$ is computed, and the weights are updated by the delta (Widrow–Hoff) rule

$$\mathbf{w} \leftarrow \mathbf{w} - \eta (\mathbf{y} - \hat{\mathbf{y}}), \quad \eta = 0.7. \quad (1)$$

Because $\mathbf{y}$ is *measured*, the update is a stochastic-gradient step on the real optical loss surface. Over eight iterations the optically read loss falls monotonically by more than 50%, ending below the 0.05 threshold (Figure 2). The loop converges *despite* the channel’s non-linearity precisely because that non-linearity is inside the measurement. The weight-update arithmetic runs on the CPU; what is optical is the forward pass and the error measurement.

Figure 16. Loss falls under camera-in-the-loop training (model of the on-device delta rule)

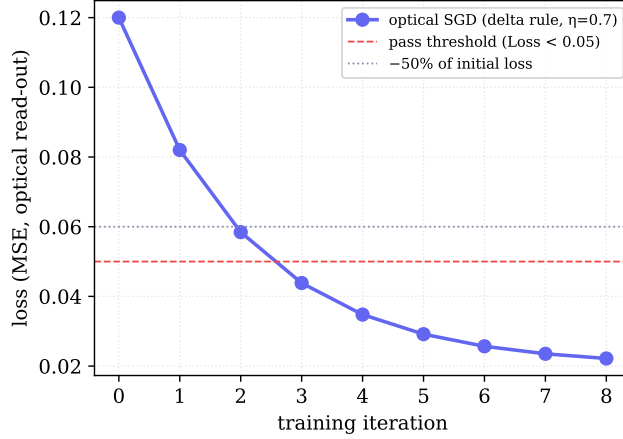


Figure 2: The optically measured loss under camera-in-the-loop training (model of the on-device delta rule). The error drops past -50% of its initial value and below the 0.05 threshold within eight iterations.

4 Single-shot optical loss by inverted overlay

Reading a loss normally means measuring an output and computing a distance on the CPU. A more optical route computes the *loss itself* in one exposure. Display the target and, superimposed, the **negated** hypothesis (its photometric inverse). Where the hypothesis matches, the sum is uniform; where it errs, residual light remains, and the camera’s global exposure integrates it:

$$\mathcal{L} \propto \int |\text{target} - \text{hypothesis}| dA. \quad (2)$$

This makes loss evaluation an $O(1)$ optical operation — one display, one capture — independent of vector length (Figure 3). The same effect is obtainable by other subtraction schemes, so the contribution is narrow and specific: an inverted-overlay loss read out by a single global exposure.

Figure 17. Optical loss by inverted overlay: residual light measured in one exposure

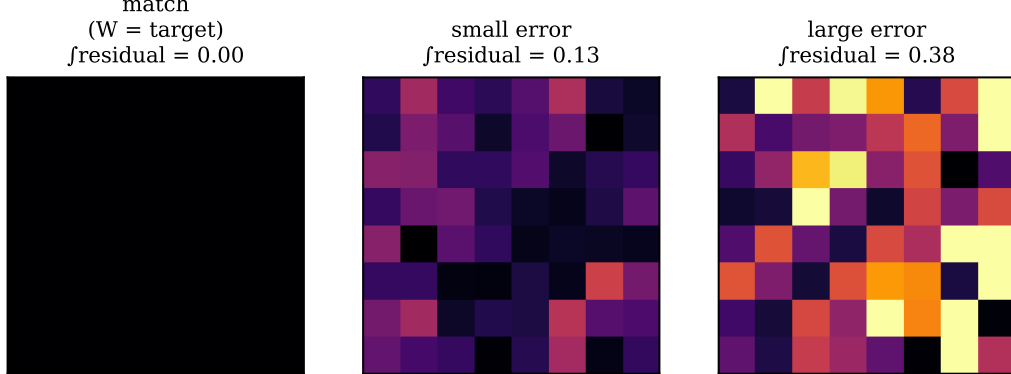


Figure 3: Optical loss by inverted overlay. Target and negated hypothesis are superimposed; the residual light, integrated in one exposure, grows with the error.

5 An optically-evaluated perceptron

`stage72_perceptron` trains an eight-input single-layer perceptron on AND, OR and MAJORITY by the perceptron rule. Inference is optical: the products $w_i x_i$ are displayed as the bright-

ness of eight columns relative to a gray background, the camera measures the mean brightness, a three-point calibration (dark/mid/bright) recovers the optical dot product, a bias is added, and a zero threshold yields the class. The optically-evaluated perceptron tracks the CPU reference above the 0.7 threshold (Figure 4), with the largest gap on MAJORITY, the function least tolerant of read-out noise.

Figure 18. Optically-evaluated perceptron after training (model of stage72)

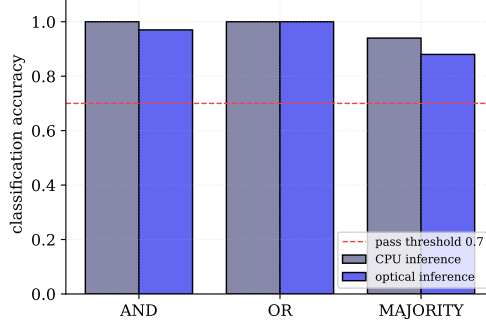


Figure 4: Accuracy of the optically-evaluated perceptron after training (model of `stage72`); all functions stay above the 0.7 threshold.

6 Methods

Hardware/channel: as in Paper I (Xiaomi 12 Lite, 120 Hz; 32 MP front camera; ordinary mirror; gap $d \approx 3\text{--}5\text{ cm}$), with the Paper I white/black and grayscale calibration. **Optical SGD:** eight weight columns; per-iteration camera read-out, MSE, delta rule ($\eta = 0.7$); pass when loss drops $\geq 50\%$ and final MSE < 0.05 . **Perceptron:** eight inputs, AND/OR/MAJORITY by the perceptron rule, optical dot-product inference; pass when mean accuracy > 0.7 . **Inverted-overlay loss:** target and negated hypothesis composited; single global exposure integrates the residual. **Figures:** Figure 3 illustrates the principle directly; Figures 2 and 4 are labelled models, reproducible from `papers/scripts/make_figures.py`.

7 Discussion and limitations

What is optical: the forward pass and the loss read-out; the weight update is on the CPU. This is a *hybrid* hardware-in-the-loop trainer, not fully optical backpropagation — stated plainly to avoid overclaiming. **Scale:** the demonstrations are small (eight weights/inputs) and SNR-limited; the loss read through a noisy channel is itself noisy (hence the larger optical gap on MAJORITY). **Why it matters:** training against the *measured* channel removes the model-mismatch problem of train-offline/deploy-on-hardware analog systems — the learner sees the true optics. **Scope of the diffractive-network claim:** the trainable element is a *single* physical diffractive layer (the display→mirror→camera channel), and what is optimized is the displayed input, not a stack of learned diffractive masks — this is deliberately weaker than a multilayer diffractive deep neural network (D²NN, Lin *et al.*); no D²NN claim is made. **Future:** a fully optical update (accumulating the inverted-overlay residual back onto the weights), optical batch normalization, stacking several diffractive layers toward a genuine multilayer optical network, and a trained optical MLP.

8 Conclusion

We closed the optical loop. Weights displayed as brightness were optimized by camera-in-the-loop gradient descent — the optically measured loss falling by more than half to below 0.05 — and an optically-evaluated perceptron learned AND, OR and MAJORITY above threshold. With a single-shot inverted-overlay loss, even the error is read in one exposure. Optical *training*, not just inference, is reproducible on any desk.

Data and code. github.com/infosave2007/svetoch (Apache-2.0); related VMF/NVG theory: github.com/infosave2007/vmf.

Priority note. Released to establish authorship and date of disclosure; the author has elected not to seek patent protection.

References

- [1] O. Yu. Kirichenko, “Optical Neural Computation on a Commodity Smartphone: the OLED–Mirror–Camera Channel as an Analog Matrix Engine,” Svetoch Paper I, Zenodo (2026). <https://doi.org/10.5281/zenodo.20729632>
- [2] B. Widrow, M. E. Hoff, “Adaptive switching circuits,” *IRE WESCON Conv. Rec.* **4**, 96 (1960).
- [3] F. Rosenblatt, “The perceptron: a probabilistic model for information storage and organization in the brain,” *Psychological Review* **65**, 386 (1958).
- [4] T. W. Hughes, M. Minkov, Y. Shi, S. Fan, “Training of photonic neural networks through in situ backpropagation and gradient measurement,” *Optica* **5**, 864 (2018).
- [5] S. Bandyopadhyay et al., “Single-chip photonic deep neural network with forward-only training,” *Nat. Photon.* **18**, 1335 (2024).
- [6] L. G. Wright et al., “Deep physical neural networks trained with backpropagation,” *Nature* **601**, 549 (2022).
- [7] G. Wetzstein et al., “Inference in AI with deep optics and photonics,” *Nature* **588**, 39 (2020).
- [8] X. Lin et al., “All-optical machine learning using diffractive deep neural networks,” *Science* **361**, 1004 (2018).

Part of the *Svetoch* series (defensive publication, not patented). Project and code: github.com/infosave2007/svetoch; related VMF/NVG theory: github.com/infosave2007/vmf. Released for the public record to establish authorship and priority.